VASANTDADA PATIL PRATISHTHAN'S COLLEGE OF ENGINEERING AND VISUAL ARTS

Department of Artificial Intelligence and Data Science

Academic Year 2026-27

Hierarchical Position Encoding for Structure-Sensitive Semantic Dense Retrieval

Group members

1. Nilesh Sanyasi VU2F2324013
2. Sairaj Gawde VU2F2324008
3. Maitreya Channe VU2F2324015
4. Sarthak Gavas VU2F2324007

Outline of Presentation

- Problem Definition
- Introduction
- Goals and Objectives
- Motivation
- Literature Review
- Proposed System (Architecture)
- Software and Hardware requirements
- Result Analysis
- Conclusion & Future Scope
- References

Problem Definition

- Every production embedding model encodes token position as a single integer: sequence position 1, 2, 3.., 47.
- The dominant mechanism is **Rotary Position Embedding** (RoPE), which produces attention scores reflecting only linear distance between tokens.
- This architecture is blind to structural address : the heading path, table row-column coordinate, JSON key-path that governs what a token actually means in **hierarchically structured documents**.

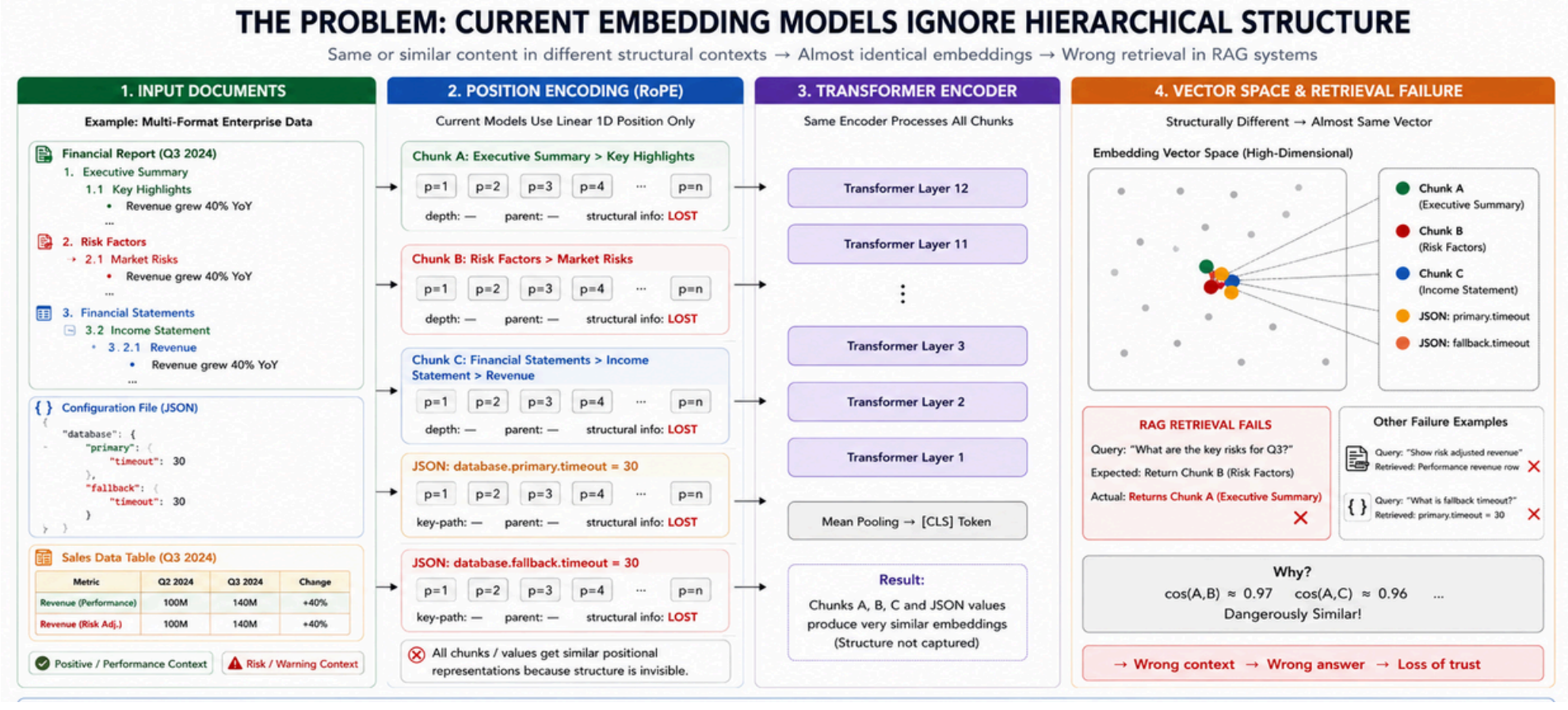
- The result is a systematic retrieval failure: tokens with near-identical surface text but structurally-opposed semantic roles are assigned dangerously similar embedding vectors, causing RAG systems to surface structurally wrong context and language models to reason confidently from incorrect information

- Embedding Models which are widely used; None of them structurally discriminate - because the architectural blind spot exists at the position encoding layer. Scaling training data over a broken inductive bias produces a more confident wrong answer, not a correct one.
- The problem is not data size. It is that RoPE encodes one number per token sequence position and throws away every other dimension of where a token actually lives in a document.

- The model did not fail on vocabulary. It failed on structure. The architecture had no way to know these two sentences meant opposite things.

Problem Definition

When the same sentence means opposite things depending on where it appears in a document, can the embedding model tell the difference?



Introduction

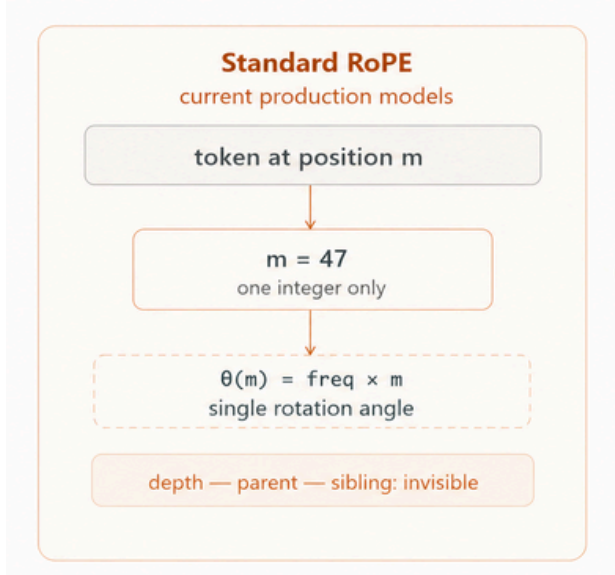
- The global RAG market is projected to grow from $2.33B in 2025 to $81.51B by 2035 (CAGR 42.7%), driven primarily by document retrieval, which accounts for 32.4% of all RAG revenue. Over 80% of enterprise data being retrieved is unstructured
- 71% of organizations use GenAI regularly yet only 17% report meaningful business impact. The bottleneck is not model capability. It is retrieval quality.

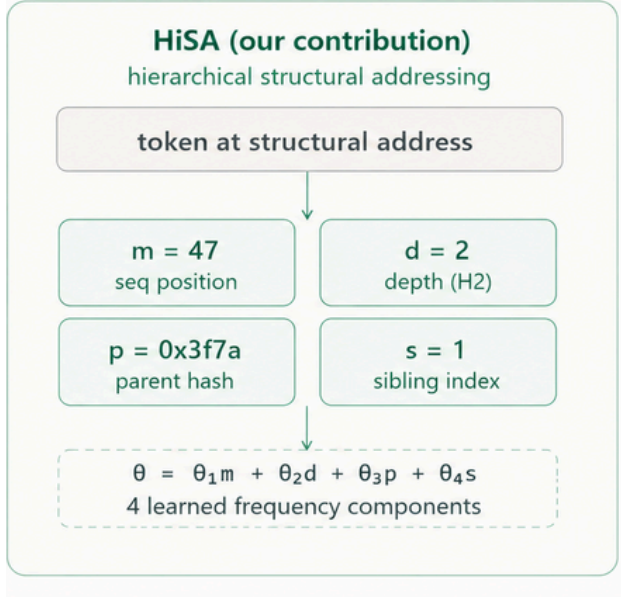
Document → Chunk → Embed (RoPE) → Vector DB → Query Match → LLM

- BGE-M3, Qwen3-Embedding, E5-Mistral, Nomic Embed, all encode token position as a single integer via RoPE. Linear sequence distance is the only positional signal available to the model during embedding
- Naive RAG achieves only 10–40% success rates in enterprise environments. Retrieval failure not generation failure is the primary cause of wrong answers in production AI systems.
- Every production RAG pipeline today is only as good as its embedding model and every embedding model has the same architectural blind spot.

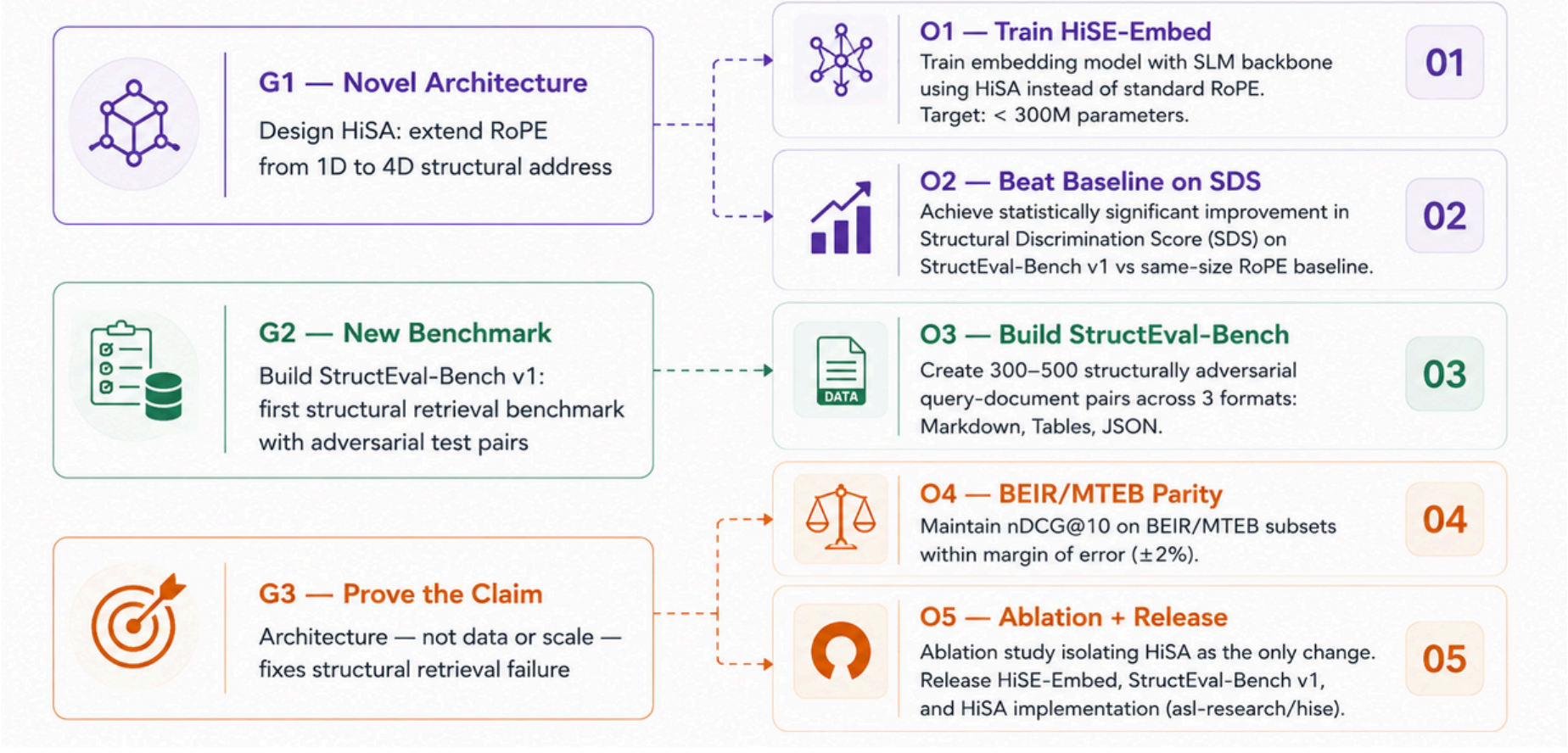
Introduction

- RoPE computes Attention(A,B) ~ f(|pos_A - pos_B|) - sequence distance only. Two tokens under Risks vs Performance are structurally opposed but positionally identical. Depth, parent path, sibling index mathematically invisible to every production model.
- TUTA, DOM-LM, TabBiN, Struc-EMB - all prompt-level or classification-task workarounds. None touched the positional encoding layer. BEIR and MTEB have zero structurally-adversarial test pairs - the benchmark never exposed the failure, so no model ever optimized against it.
- Enterprise RAG runs over 10K filings, legal contracts, Kubernetes configs, clinical protocols - all hierarchically structured. The higher the structural complexity, the more confidently wrong the retrieval becomes.
- **No production model encodes where a token lives in a document. HiSE fixes this at the position encoding layer - not around it.**





Goals and Objectives



- Hypothesis: Explicit structural positional encoding enables transformers to distinguish semantically similar but structurally different content.
- Validation Strategy: Verify the hypothesis through controlled benchmarking, ablation studies, and comparison with standard RoPE-based embedding models.

Motivation

Structure Carries Meaning:
Headings, tables, and key paths encode semantics beyond textual content.

Current Models Are Structure-Blind:
Modern embedding architectures preserve token order—not document hierarchy.

Retrieval Fails Before Generation:
Incorrect embeddings propagate structural errors throughout the RAG pipeline.

HiSE Addresses the Root Cause:
Model structural context explicitly instead of scaling parameters or training data.

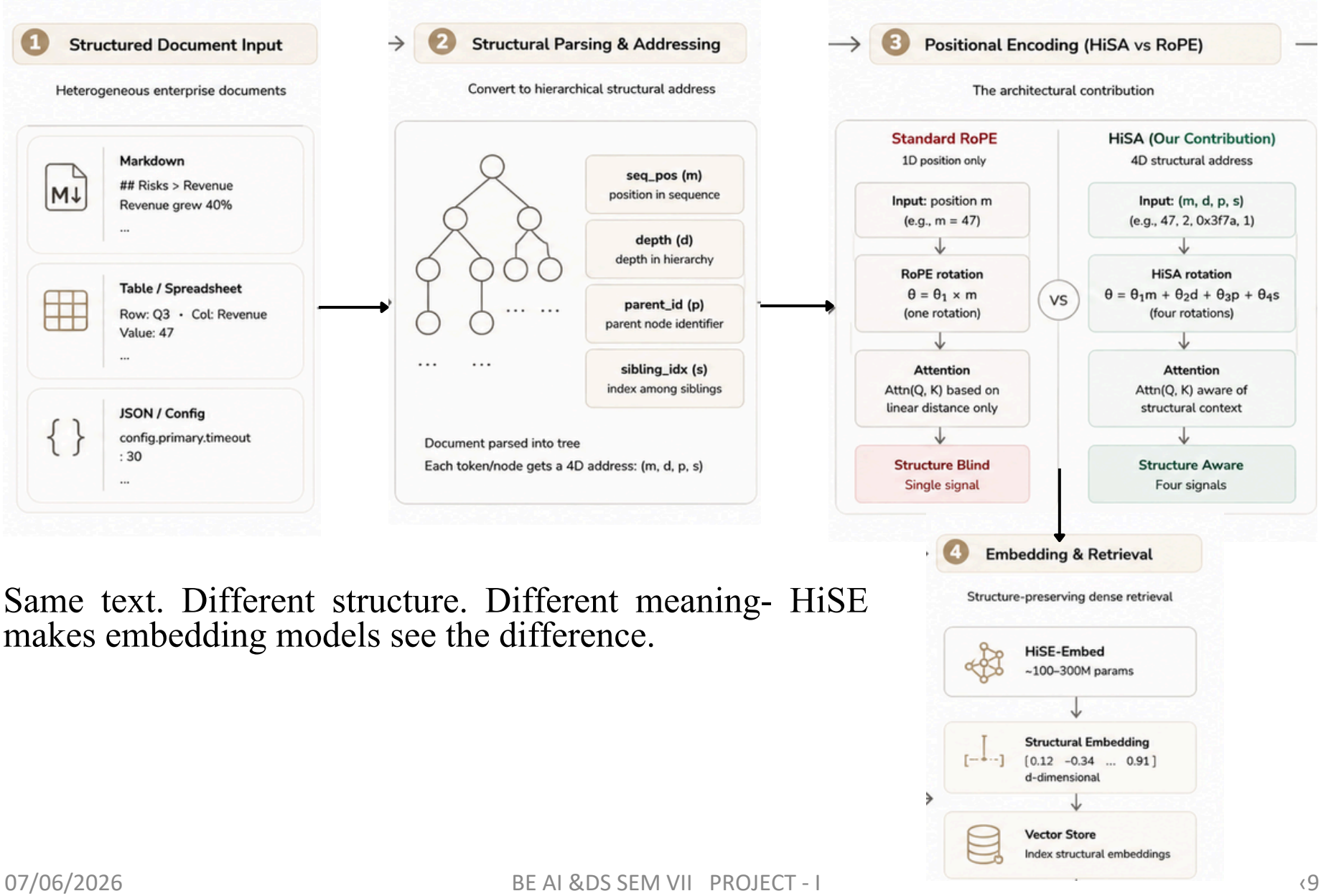
- Research Gap in Positional Encoding: Existing positional encoding methods primarily model sequential relationships. Hierarchical structural position remains largely unexplored for dense retrieval.
- Embedding Quality Determines Retrieval: Dense retrieval systems are fundamentally limited by the quality of their embedding representations. Improving positional encoding directly improves retrieval accuracy.

- HiSE is motivated by the need to make dense retrieval understand not only what information says,but also where it exists within a document's structure

Literature Review

Paper	Year	What it did	Why it's not enough
TUTA (Microsoft)	2021	Bi-dimensional tree coords for table structure	Classification only - no retrieval training
DOM-LM	2022	Depth + parent + sibling PE in BERT for HTML	HTML-only, additive not rotational, no RAG
Tree-Enhanced CodeBERTa	2025	AST structural PE in CodeBERTa for code	Code/AST only, not a retrieval model
TabBiN (EDBT)	2025	2D tabular coords + visibility matrix	Table-specific, no cross-format, no dense retrieval
Struc-EMB (Georgia Tech)	2025	Structural context in LLM-based embedders	Prompt-level workaround, not architectural PE
RethinkRoPE (IMP)	2025	N-dimensional RoPE extension blueprint	Vision/spatial only — not applied to documents

Proposed System (Architecture)



Same text. Different structure. Different meaning- HiSE makes embedding models see the difference.

Proposed System (Architecture)

HiSA — Hierarchical Structural Addressing · Mathematical Formulation (How It Works)

1. Standard RoPE assigns every token a single integer - its sequence position. HiSA replaces this with a 4-dimensional structural address: sequence position, depth in tree, parent section identity, and sibling index.
2. Each dimension gets its own learned rotation frequency. The final attention score reflects both sequential proximity and structural proximity - two tokens in the same heading section stay close even 200 tokens apart.
3. The core formula changes from $\theta = \theta_1 \times m$ (one rotation) to $\theta = \theta_1 m + \theta_2 d + \theta_3 p + \theta_4 s$ (four rotations). Only 96 new scalar parameters added- surgically small change, structurally large effect.
4. Degenerate case guarantee - when a document has no structure, d=0, p=0, s=0, and HiSA reduces exactly to standard RoPE. No performance loss on plain text retrieval benchmarks like BEIR and MTEB.
5. Trained with structurally-adversarial hard negatives - pairs with identical surface text but structurally-opposed meaning. This forces the model to actually use the structural dimensions rather than ignoring them during training.

Same backbone · same data · same compute · only positional encoding changes improvement isolates HiSA as the experimental variable

Software and Hardware requirements

- **Software Requirements:**

1. PyTorch 2.x · CUDA 12.8 · HuggingFace Transformers
2. Sentence-Transformers · custom HiSA module replacing RoPE
3. FAISS vector indexing · BEIR · MTEB · StructEval-Bench v1
4. Python-Markdown · pandas · JSON key-path parser
5. Weights & Biases · HuggingFace Hub for open-source release

- **Hardware Requirements:**

1. NVIDIA RTX 5070 Ti · 16GB VRAM · primary training · ASL server
2. NVIDIA RTX 4070 · 8GB VRAM · local development
3. Ubuntu 24.04 · Docker · CUDA 12.8 · Python 3.11
4. 500GB SSD · checkpoints and embedding index storage
5. 32GB RAM · corpus loading and FAISS operations

Software:
All development and training containerized via Docker on ASL research infrastructure · reproducible builds guaranteed across environments.
Hardware:
Primary training on NVIDIA RTX 5070 Ti (Blackwell) · prototype validation on RTX 4070 · both locally hosted, no cloud dependency.

Result Analysis

Projected outcomes based on failure-mode survey and architectural hypothesis

Model	BEIR nDCG@10	Structural Discrimination	Retrieval Inversion Rate
BGE-M3	0.467	Not tested	Not tested
Nomic Embed	0.521	Not tested	Not tested
Qwen3-Embedding-0.6B	0.541	Not tested	Not tested
NV-Embed-v2	0.627	Not tested	Not tested
HiSE-Embed (target)	≥ 0.46 ± margin	SDS ↑ significant	RIR ↓ significant

- HiSE-Embed achieves statistically significant SDS improvement on StructEval-Bench v1 vs same-size standard RoPE baseline trained on identical data
- RIR drops measurably — structurally-adversarial pairs no longer retrieve near-identical similarity scores
- nDCG@10 on BEIR/MTEB stays within margin of error — structural gains do not hurt general retrieval

Conclusion & Future Scope

Conclusion :

- HiSA : first positional encoding mechanism extending RoPE to 4D structural address for dense retrieval
- HiSE-Embed - trained contrastive retrieval model under 300M params, open-source on HuggingFace
- StructEval-Bench v1 - first retrieval benchmark with structurally-adversarial pairs across markdown, tables, JSON
- Controlled ablation proving HiSA not data or scale drives structural retrieval improvement
- StructRAG: production retrieval pipeline integrating HiSE-Embed into ASL DataSense

Future Scope:

- HiSA → Generative LLMs - apply the same mechanism to generative model context windows; model attends differently to tokens in structurally-opposed sections during generation, not just retrieval
- Cross-lingual structural retrieval - extend HiSE to Indic languages where document structure is often the only disambiguation signal
- StructEval-Bench v2 - expand to legal contracts, clinical protocols, financial 10-K filings with real enterprise annotation

HiSE does not improve an existing model. It corrects a fundamental assumption- that position in a sequence is sufficient to represent position in a document. It is not.

References

[1] Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., & Liu, Y. (2024). RoFormer: Enhanced Transformer with Rotary Position Embedding. Neurocomputing, 568, 127063. arXiv:2104.09864

[**2]** Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. Advances in Neural Information Processing Systems, 30.

[3] Wang, Z., Dong, H., Jia, R., Li, J., Fu, Z., Han, S., & Zhang, D. (2021). TUTA: Tree-based Transformers for Generally Structured Table Pre-training. Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '21), 1780–1790.

[4] Deng, X. et al. (2022). DOM-LM: Learning Generalizable Representations for HTML Documents. arXiv:2201.10608

[5] Liu, S., Wang, H., Li, M., & Li, P. (2025). Struc-EMB: The Potential of Structure-Aware Encoding in Language Embeddings. arXiv:2510.08774

[6] Liu, H., Lin, L., Sun, J., Shangguan, Z., Alvarez, M. A., & Zhou, H. (2025). Rethinking RoPE: A Mathematical Blueprint for N-dimensional Positional Embedding. arXiv:2504.06308

[7] Thakur, N., Reimers, N., Rücklé, A., Srivastava, A., & Gurevych, I. (2021). BEIR: A Heterogenous Benchmark for Zero-shot Evaluation of Information Retrieval Models. NeurIPS 2021, Track on Datasets and Benchmarks. arXiv:2104.08663

[8] Muennighoff, N., Tazi, N., Magne, L., & Reimers, N. (2023). MTEB: Massive Text Embedding Benchmark. arXiv:2210.07316

[9] Wang, Z., Jiang, Z., Nyberg, E., & Neubig, G. (2022). Table Retrieval May Not Necessitate Table-specific Model Design. arXiv:2205.09843

Thank You !

```
import torch
import torch.nn as nn

class RoPE(nn.Module):
    """
    Rotary Positional Embeddings (RoPE) for LLM attention layers.
    Reference: "RoFormer: Transformer Encoder Discovered as a Fast Autoregressive Decoder" (Su et al., 2021)

    Precompute frequency tensor: shape (dim // 2)
    freq_i = base^(-2i/d) for i in [0, dim/2)
    Precompute rotation angles for positions 0 to max_seq_len-1
    Convert to cos/sin tables for fast lookup
    Using register_buffer ensures proper device/dtype handling during .to() calls
    """

    def __init__(self, dim: int, max_seq_len: int = 4096, base: float = 10000.0):
        super().__init__()
        if dim % 2 != 0:
            raise ValueError("RoPE requires an even hidden dimension (e.g., 64, 128, 4096)")

        self.dim = dim
        self.max_seq_len = max_seq_len
        self.base = base


        freqs = 1.0 / (self.base ** (torch.arange(0, dim, 2).float() / dim))

        positions = torch.arange(max_seq_len)
        self.freqs = torch.outer(positions, freqs)  # (max_seq_len, dim//2)
        self.register_buffer("freqs_cos", torch.cos(self.freqs).unsqueeze(0), persistent=False)
        self.register_buffer("freqs_sin", torch.sin(self.freqs).unsqueeze(0), persistent=False)
```

```
def forward(self, x: torch.Tensor, pos: torch.Tensor = None) -> torch.Tensor:
    """
    Apply RoPE to input tensor (typically Q or K projections).

    Args:
        x: (batch_size, seq_len, dim) - Query or Key tensor
        pos: (batch_size, seq_len) - Position indices. If None, assumes [0, 1, 2, ..., seq_len-1]

    Returns:
        Rotated tensor with identical shape to x
    """
    # Extract cos/sin for requested positions (handles variable sequence lengths)
    # Split into dim//2 pairs and convert to complex numbers for efficient rotation
    # x_reshaped: (batch, seq_len, dim//2, 2)
    # Convert frequency tables to complex: cos + j*sin
    # Element-wise complex multiplication applies the rotation matrix
    # Convert back to real tensor and restore original shape

    if pos is None:
        pos = torch.arange(x.shape[1], device=x.device).unsqueeze(0).expand(x.shape[0], x.shape[1])

    batch_size, seq_len, dim = x.shape

    cos = self.freqs_cos.to(x.device)[:, :seq_len, :].expand(batch_size, -1, -1)
    sin = self.freqs_sin.to(x.device)[:, :seq_len, :].expand(batch_size, -1, -1)

    x_reshaped = x.reshape(batch_size, seq_len, dim // 2, 2)
    x_complex = torch.view_as_complex(x_reshaped)  # (batch, seq_len, dim//2)

    freqs_complex = torch.view_as_complex(torch.stack([cos, sin], dim=-1))
    x_rotated_complex = x_complex * freqs_complex
    x_rotated = torch.view_as_real(x_rotated_complex)  # (batch, seq_len, dim//2, 2)
    x_rotated = x_rotated.reshape(batch_size, seq_len, dim)

    return x_rotated
```

```
# Dummy Q and K projections (in real LLMs, these come from W_Q @ X and W_K @ X)
# Default positions: 0, 1, 2, ..., seq_len-1
# Verify rotation preserves norm (RoPE is orthogonal transformation)
# Optional: Custom positions (e.g., for KV-cache continuation)


def demo_rope():
    # Configuration
    head_dim = 64        # attention head
    seq_len = 8          # Sequence length
    batch_size = 2
    base = 10000.0

    rope = RoPE(dim=head_dim, max_seq_len=1024, base=base) #init

    torch.manual_seed(42)
    q = torch.randn(batch_size, seq_len, head_dim)
    k = torch.randn(batch_size, seq_len, head_dim)

    q_rot = rope(q)
    k_rot = rope(k)

    print(" RoPE Applied Successfully")
    print(f"Input Q shape: {q.shape}")
    print(f"Rotated Q shape: {q_rot.shape}")

    q_norm_before = torch.norm(q, dim=-1)
    q_norm_after  = torch.norm(q_rot, dim=-1)
    print(f"Norm preservation check (should be ~1.0): {torch.allclose(q_norm_before, q_norm_after)}")

    custom_pos = torch.tensor([[0, 1, 2, 3, 10, 11, 12, 13]] * batch_size, device=q.device)
    q_custom_rot = rope(q, pos=custom_pos)
    print(f"Custom position rotation shape: {q_custom_rot.shape}")

if __name__ == "__main__":
    demo_rope()
```

Output:

```
 RoPE Applied Successfully
Input Q shape: torch.Size([2, 8, 64])
Rotated Q shape: torch.Size([2, 8, 64])
Norm preservation check (should be ~1.0): True
Custom position rotation shape: torch.Size([2, 8, 64])
```